

BCA182 · EMBEDDED SYSTEMS PROGRAMMING

# Real-Time Multisensor Room Monitor

A concurrent STM32F103 FreeRTOS node that measures temperature, humidity, ambient light, and occupancy — driving an SSD1306 navigation interface, a 1 kHz temperature safety alarm, and an ACTIVE/INACTIVE power state machine. Verified by 13 automated unit tests with zero static-analysis defects.

Perch Arnel II Montefalcon · Mindanao State  
University – Iligan Institute of Technology

---

# Table of Contents

---

|                                                                 |           |
|-----------------------------------------------------------------|-----------|
| Student & Course Information . . . . .                          | 3         |
| <b>1. Problem and Requirements</b>                              | <b>3</b>  |
| 1.1 Problem Statement . . . . .                                 | 3         |
| 1.2 Functional Requirements Traceability . . . . .              | 3         |
| <b>2. System Architecture and Design</b>                        | <b>4</b>  |
| 2.1 Hardware Block Diagram & Pinout . . . . .                   | 4         |
| 2.2 Simulated Circuit Schematic Diagram . . . . .               | 5         |
| 2.3 Circuit Wiring Netlist . . . . .                            | 5         |
| 2.4 System State Machine . . . . .                              | 6         |
| 2.3 Software Architecture & Subsystem Decomposition . . . . .   | 6         |
| <b>3. FreeRTOS Architecture</b>                                 | <b>6</b>  |
| 3.1 Mandatory FreeRTOS Task Table . . . . .                     | 6         |
| 3.2 Priority Justification: Scheduling Urgency . . . . .        | 7         |
| 3.3 Task States Analysis . . . . .                              | 8         |
| 3.4 Inter-Task Communication (IPC) Mechanisms . . . . .         | 8         |
| 3.5 Required Explanations . . . . .                             | 8         |
| <b>4. Implementation Details</b>                                | <b>9</b>  |
| 4.1 Peripheral Interfacing . . . . .                            | 9         |
| <b>5. Verification and Testing</b>                              | <b>10</b> |
| 5.1 Automated Unit Testing (pio test -e native) . . . . .       | 10        |
| 5.2 Functional Verification in Wokwi (FT-01 to FT-10) . . . . . | 10        |
| 5.3 Deliberate FreeRTOS Fault Experiments . . . . .             | 11        |
| <b>6. Static Code Analysis (pio check)</b>                      | <b>12</b> |
| 6.1 Findings and Corrective Actions . . . . .                   | 12        |
| 6.2 Final Analysis Result . . . . .                             | 13        |
| <b>7. Engineering Discussion</b>                                | <b>13</b> |
| 7.1 Wokwi Simulation vs. Physical Hardware Deep-Dive . . . . .  | 13        |
| 7.2 Memory Allocation & Sizing . . . . .                        | 14        |
| <b>8. Conclusion</b>                                            | <b>14</b> |
| Key Accomplishments: . . . . .                                  | 14        |

# Laboratory Activity No. 1: Real-Time Multisensor Room Monitoring System

Mindanao State University – Iligan Institute of Technology College of Computer Studies  
Department of Computer Applications BCA182: Embedded Systems Programming

## Student & Course Information

| Field                   | Details                                                                                       |
|-------------------------|-----------------------------------------------------------------------------------------------|
| Student Name            | Perch Arnel II Montefalcon                                                                    |
| Course & Section        | BCA182 – Embedded Systems Programming                                                         |
| Instructor / Evaluator  | Paul Rodolf P. Castor (paulrodolf.castor@g.msuiit.edu.ph)                                     |
| Activity Title          | Laboratory Activity No. 1: Real-Time Multisensor Room Monitoring System                       |
| Target Platform         | STM32 Blue Pill (STM32F103C8T6, ARM Cortex-M3 @ 72 MHz)                                       |
| Development Environment | PlatformIO Core 6.2.0, STM32Cube HAL Framework, Wokwi Virtual Simulator                       |
| RTOS                    | FreeRTOS Kernel v10.3.1 (Thread-Mode Direct-Yield Port, 20 Hz Tick)                           |
| Public Repository       | <a href="https://github.com/Not-to-Shabby/BCA182">https://github.com/Not-to-Shabby/BCA182</a> |

## 1. Problem and Requirements

### 1.1 Problem Statement

Modern indoor environments require autonomous, real-time monitoring of environmental parameters (temperature, humidity, ambient light) and occupancy to ensure occupant comfort, safety, and energy efficiency. Traditional super-loop (bare-metal `while(1)`) embedded architectures suffer from severe timing drift, blocking sensor delays, unresponsive user input, and unpredictable task execution.

This laboratory requires designing, implementing, verifying, and documenting a concurrent, multi-tasking room monitoring node on the STM32F103C8T6 microcontroller using the **STM32Cube framework** and **native FreeRTOS primitives**. Arduino frameworks, libraries, and abstractions are strictly prohibited.

### 1.2 Functional Requirements Traceability

| ID    | Requirement             | Engineering Specification                                                                                                      | Implementation Module               |
|-------|-------------------------|--------------------------------------------------------------------------------------------------------------------------------|-------------------------------------|
| FR-01 | Temperature Measurement | Periodically sample ambient temperature via DHT22. Operating limits: $18.0^{\circ}\text{C} \leq T \leq 30.0^{\circ}\text{C}$ . | <code>sensors.c</code> (SensorTask) |
| FR-02 | Humidity Measurement    | Periodically sample relative humidity (0.0% - 100.0%) via DHT22.                                                               | <code>sensors.c</code> (SensorTask) |

| ID    | Requirement               | Engineering Specification                                                                                         | Implementation Module      |
|-------|---------------------------|-------------------------------------------------------------------------------------------------------------------|----------------------------|
| FR-03 | Ambient-Light Monitoring  | Continuously acquire ambient light via photoresistor (LDR) on ADC1_IN0, scaled to 0–100%.                         | ldr.c, sensors.c           |
| FR-04 | Motion Detection          | Detect human motion using a digital PIR motion sensor on PA3.                                                     | motion.c (MotionTask)      |
| FR-05 | OLED Display              | Exclusively display one selected measurement metric at a time on SSD1306 (128x64 I2C).                            | display.c (DisplayTask)    |
| FR-06 | Rotary Encoder Navigation | Switch views cyclically via rotary encoder: Temp ↔ Humidity ↔ Light ↔ Motion.                                     | input.c (InputTask)        |
| FR-07 | Temperature Alarm         | Activate active buzzer on PA2 when $T < 18.0^{\circ}\text{C}$ (TOO_COLD) or $T > 30.0^{\circ}\text{C}$ (TOO_HOT). | alarm.c (AlarmTask)        |
| FR-08 | Activity States           | System operates in distinct ACTIVE and INACTIVE states.                                                           | system_state.c (StateTask) |
| FR-09 | Automatic Inactivity      | Transition from ACTIVE to INACTIVE (OLED standby) after 15 s without PIR motion.                                  | system_state.c (StateTask) |
| FR-10 | Automatic Reactivation    | PIR motion or rotary encoder interaction immediately restores ACTIVE state.                                       | motion.c, input.c          |

## 2. System Architecture and Design

### 2.1 Hardware Block Diagram & Pinout

```

+-----+
|               STM32F103C8T6 (Blue Pill)               |
+-----+
| PA0 (ADC1_IN0) <---- Analog Voltage ---- LDR Sensor |
| PA1 (GPIO O.D) <---> Single-Wire    <---> DHT22      |
| PA2 (TIM2_CH3) <---> PWM Tone (1kHz) --> Active Buzzer|
| PA3 (GPIO In)  <---- Digital Motion <--- PIR Sensor  |
| PA4 (EXTI4)    <---- Quadrature CLK <-- KY-040 Enc.  |
| PA5 (GPIO In)  <---- Quadrature DT  <-- KY-040 Enc.  |
| PB6 (I2C1_SCL) <---> I2C Clock  <----- SSD1306 OLED |
| PB7 (I2C1_SDA) <---> I2C Data  <-----> SSD1306 OLED |
| PA9 (USART1_TX) <---> Serial Monitor (115200 8N1)    |
| PA10(USART1_RX) <---- Serial Terminal                 |
| PC13 (GPIO Out) <---> Heartbeat LED (Active LOW)      |
+-----+

```

## 2.2 Simulated Circuit Schematic Diagram

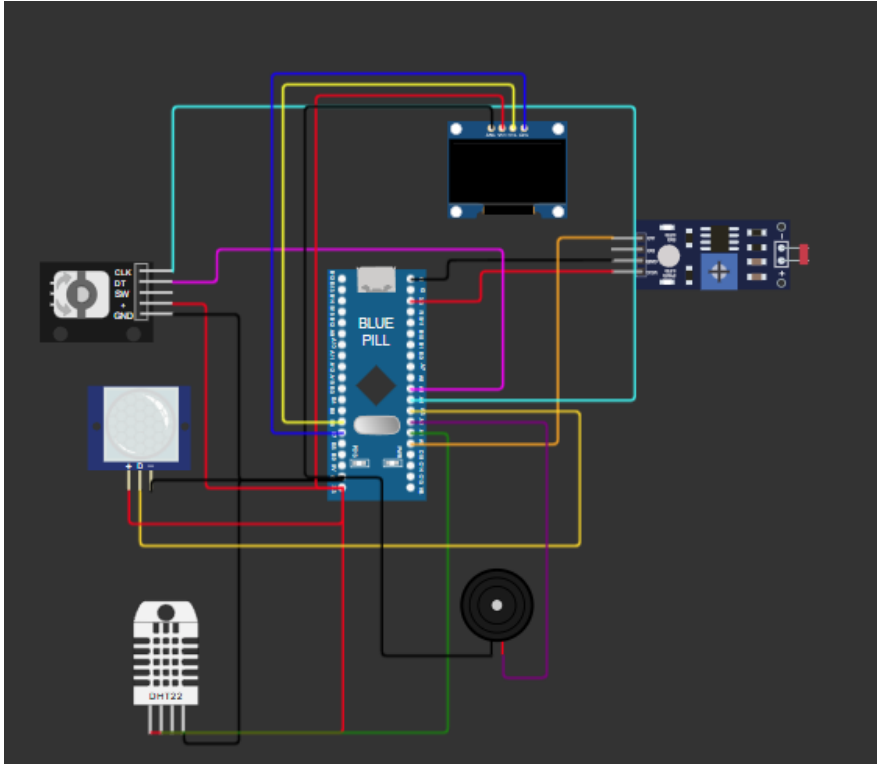


Figure 1: Wokwi circuit simulation diagram displaying physical sensor connections to the STM32 Blue Pill.

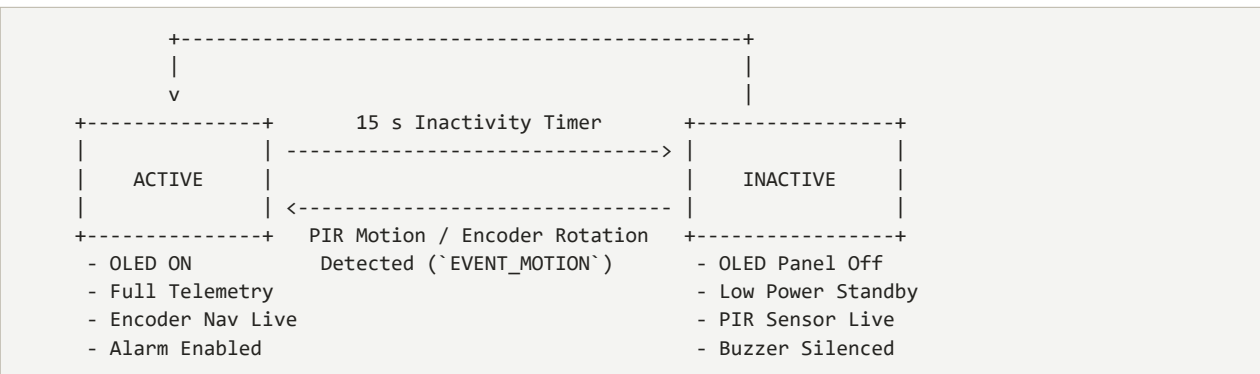
## 2.3 Circuit Wiring Netlist

| Source Pin (STM32) | Target Pin (Component)                                           | Wire Color | Signal Type              | Description                               |
|--------------------|------------------------------------------------------------------|------------|--------------------------|-------------------------------------------|
| 3V3.1              | oled1:VCC, dht1:VCC, ldr1:VCC, pir1:VCC, encoder1:VCC            | Red        | Power (+3.3V)            | Main regulated 3.3V DC power rail         |
| GND.1              | oled1:GND, dht1:GND, ldr1:GND, pir1:GND, encoder1:GND, buzzer1:2 | Black      | Ground (0V)              | Common system reference ground            |
| PA0                | ldr1:A0                                                          | Orange     | Analog In (ADC1_IN0)     | Ambient light sensor analog voltage       |
| PA1                | dht1:SDA                                                         | Green      | Bidirectional Open-Drain | DHT22 single-wire digital communications  |
| PA2                | buzzer1:1                                                        | Purple     | Digital Out (TIM2_CH3)   | 1 kHz audible PWM alarm signal            |
| PA3                | pir1:OUT                                                         | Gold       | Digital In (Pulldown)    | Human presence motion pulse               |
| PA4                | encoder1:CLK                                                     | Cyan       | Digital In (EXTI4)       | Rotary encoder quadrature clock interrupt |
| PA5                | encoder1:DT                                                      | Magenta    | Digital In (Pull-up)     | Rotary encoder quadrature direction line  |

| Source Pin (STM32) | Target Pin (Component) | Wire Color | Signal Type                 | Description                                |
|--------------------|------------------------|------------|-----------------------------|--------------------------------------------|
| PB6                | oled1:SCL              | Yellow     | Alternate Function (I2C1)   | Hardware I2C Clock (400 kHz Fast Mode)     |
| PB7                | oled1:SDA              | Blue       | Alternate Function (I2C1)   | Hardware I2C Data line (400 kHz Fast Mode) |
| PA9                | \$serialMonitor:RX     | Amber      | Alternate Function (USART1) | Serial diagnostic logging (115200 8N1)     |
| PA10               | \$serialMonitor:TX     | Amber      | Input                       | Virtual serial monitor input               |

## 2.4 System State Machine

The system implements an automated power-management state machine:



## 2.3 Software Architecture & Subsystem Decomposition

The software is organized strictly into independent modules separating hardware-independent decision logic from peripheral drivers:

- `logic.c / logic.h`: Pure functions containing zero hardware dependencies:
- `evaluateTemperature(float)`: Boundary decision logic for normal, low, and high temperature.
- `nextDisplayMode()` / `previousDisplayMode()`: Cyclic enumeration traversal with bidirectional wraparound.
- `evaluateSystemState()`: Deterministic timeout and motion state transitions.
- `rtos_objects.c / rtos_objects.h`: Centralized instantiation of FreeRTOS queues, recursive mutexes, and event groups.
- **Hardware Drivers**: `dht22.c`, `ldr.c`, `buzzer.c`, `ssd1306.c`, `input.c`, `motion.c`.

# 3. FreeRTOS Architecture

## 3.1 Mandatory FreeRTOS Task Table

| Task Name   | Responsibility                                        | Execution Trigger / Period                   | Assigned Priority | IPC Mechanism Used                                   | Blocked State Condition             |
|-------------|-------------------------------------------------------|----------------------------------------------|-------------------|------------------------------------------------------|-------------------------------------|
| MotionTask  | Sample PIR sensor on PA3; detect movement edges       | Periodic (100 ms)                            | 3 (High)          | systemEvents (EVENT_MOTION, EVENT_PIR_LEVEL)         | vTaskDelay                          |
| StateTask   | Track 15 s inactivity timer; manage sleep/wake states | Event-driven / Periodic (250 ms)             | 3 (High)          | systemEvents (EVENT_ACTIVE, EVENT_MOTION)            | xEventGroupWaitBits                 |
| InputTask   | Process rotary encoder rotation; cycle display page   | Periodic (50 ms)                             | 3 (High)          | displayModeQueue, systemEvents                       | vTaskDelay                          |
| SensorTask  | Acquire DHT22 and LDR; push to consumer queues        | Periodic (vTaskDelayUntil, 2000 ms)          | 2 (Medium)        | sensorToDisplayQueue, sensorToAlarmQueue             | vTaskDelayUntil                     |
| AlarmTask   | Evaluate temperature; actuate active buzzer on PA2    | Event-driven (Queue Receive, 250 ms timeout) | 2 (Medium)        | sensorToAlarmQueue, systemEvents (EVENT_ALARM)       | xQueueReceive                       |
| DisplayTask | Exclusively own and refresh SSD1306 OLED display      | Event / Periodic (100 ms)                    | 1 (Low)           | sensorToDisplayQueue, displayModeQueue, systemEvents | xQueueReceive / xEventGroupWaitBits |

### 3.2 Priority Justification: Scheduling Urgency

Task priorities were assigned strictly based on **acceptable latency and scheduling urgency**:

#### 1. Priority 3 (Urgent Responsiveness):

- **InputTask**: User rotation of the physical knob must register immediately without perceived lag or missed quadrature pulses.
- **MotionTask & StateTask**: Immediate detection of human occupancy and state switching prevents sluggish wake-up behavior.

#### 2. Priority 2 (Periodic Telemetry & Safety):

- **SensorTask**: Environmental room conditions change slowly; a 2000 ms period is optimal. Running at Priority 2 ensures it preempts UI rendering to maintain exact sampling intervals.
- **AlarmTask**: Must promptly evaluate fresh sensor readings to activate safety alerts.

#### 3. Priority 1 (Background Presentation):

- **DisplayTask**: Human visual persistence accommodates tens of milliseconds of rendering latency. Framebuffer streaming over I2C takes significant bus time; placing it at Priority 1 prevents display updates from starving input handling or sensor timing.

### 3.3 Task States Analysis

During system operation, tasks transition through the following states:

- **Running:** The single task currently executing on the Cortex-M3 core (e.g., SensorTask while sampling ADC).
- **Ready:** Tasks capable of execution but waiting for CPU allocation because an equal or higher priority task is Running.
- **Blocked:** Tasks suspended waiting for an external event or timeout:
  - SensorTask blocks inside vTaskDelayUntil awaiting its 2000 ms period.
  - DisplayTask blocks inside xQueueReceive awaiting fresh sensor frames or inside xEventGroupWaitBits during INACTIVE sleep.
  - AlarmTask blocks inside xQueueReceive awaiting updated temperature data.
- **Suspended:** No tasks are permanently suspended in this architecture.

### 3.4 Inter-Task Communication (IPC) Mechanisms

#### 1. FreeRTOS Queues:

- sensorToDisplayQueue (Length 1, item size sizeof(SensorData)): Transmits fresh measurements from SensorTask to DisplayTask. Length 1 with xQueueOverwrite() ensures consumers always receive the latest environmental snapshot without buffer bloat.
- sensorToAlarmQueue (Length 1, item size sizeof(SensorData)): Decouples alarm processing from display rendering.
- displayModeQueue (Length 1, item size sizeof(DisplayMode)): Delivers navigation selections from InputTask to DisplayTask.

#### 2. FreeRTOS Recursive Mutex (serialMutex):

- Protects the shared USART1 peripheral. Since multiple tasks (SensorTask, InputTask, AlarmTask, StateTask) output diagnostic telemetry, concurrent uncoordinated writes would interleave characters and corrupt serial logs. Log\_Begin() acquires serialMutex and Log\_End() releases it.

#### 3. FreeRTOS Event Group (systemEvents):

- EVENT\_ACTIVE (BIT0): High when system is active; cleared on 15 s timeout.
- EVENT\_MOTION (BIT1): Pulsed by MotionTask or InputTask to trigger immediate wake-up.
- EVENT\_ALARM (BIT2): High when temperature is out of normal bounds (18.0°C - 30.0°C).
- EVENT\_PIR\_LEVEL (BIT3): Reflects live logic level of the PIR sensor for telemetry packets.

### 3.5 Required Explanations

#### Explanation of vTaskDelayUntil() vs vTaskDelay() (Section 23)

- vTaskDelay(pdMS\_TO\_TICKS(2000)) specifies a delay relative to the moment the function is invoked. If sensor acquisition and communication take 35 ms, the total cycle period becomes 2000 ms + 35 ms = 2035 ms. Over 100 iterations, the sampling phase drifts by 3.5 seconds. Any task preemption further increases this timing error.

- `vTaskDelayUntil(&lastWakeTime;, pdMS_TO_TICKS(2000))` computes the unblock time relative to `lastWakeTime`, the absolute tick count of the previous period. The kernel automatically accounts for the execution time of sensor reading and any preemption by higher-priority tasks, ensuring the sampling frequency remains strictly 0.5 Hz with zero cumulative phase drift.

### Explanation of Shared Resource Protection & Mutex (Section 37)

- **Shared Resource:** The hardware USART1 transmit data register (`USART1->DR`) and status flags (`USART1->SR`).
- **Competing Tasks:** `SensorTask` (periodic sensor telemetry), `InputTask` (encoder page logs), `MotionTask` (occupancy alerts), `AlarmTask` (limit trip notifications), and `StateTask` (sleep/wake mode logs).
- **Failure Mode Prevented:** Without `serialMutex`, if `SensorTask` is preempted mid-sentence by `InputTask`, the output stream becomes garbled (e.g., `[Sens[Encoder] View: Temp or] Temp: 25.0 C`), corrupting automated serial parsers and human debugging consoles.

## 4. Implementation Details

---

### 4.1 Peripheral Interfacing

#### 1. SSD1306 OLED (Hardware I2C1):

- Configured on pins **PB6 (SCL)** and **PB7 (SDA)** in Alternate Function Open Drain mode (`GPIO_MODE_AF_OD`).
- Driven at 400 kHz Fast Mode via STM32 HAL (`HAL_I2C_Mem_Write`). Page-by-page streaming sends the 1024-byte framebuffer in 128-byte chunks to control byte `0x40`.

#### 2. DHT22 Single-Wire Sensor:

- Interfaced on **PA1**. The protocol requires sub-microsecond pulse timing: a 1.2 ms low start pulse, followed by listening for the sensor's 80  $\mu$ s response and 40 data bits.
- Microsecond delays and pulse width measurements are derived from the Cortex-M **DWT cycle counter** (`DWT->CYCCNT`) running at core clock frequency.

#### 3. LDR Photoresistor (ADC1):

- Wired to **PA0 (ADC1\_IN0)**. Configured for 12-bit right-aligned conversion. Sampled via `HAL_ADC_PollForConversion()`. The raw value (0–4095) is inverted and normalized to 0–100% relative brightness.

#### 4. KY-040 Rotary Encoder:

- CLK on **PA4** configured with `GPIO_MODE_IT_FALLING` (`EXTI4_IRQn`).
- DT on **PA5** sampled in the EXTI callback to determine rotation direction. Steps are accumulated in a volatile counter and consumed by `InputTask`.

#### 5. Active Buzzer (TIM2 PWM):

- Interfaced on **PA2** (TIM2 Channel 3). Configured for 50% duty cycle PWM at 1 kHz (1 MHz / 1000). Turned on and off via `HAL_TIM_PWM_Start()` and `HAL_TIM_PWM_Stop()`.

## 6. PIR Motion Sensor:

- Digital input on **PA3** with internal pulldown. Sampled every 100 ms by MotionTask.

# 5. Verification and Testing

## 5.1 Automated Unit Testing (pio test -e native)

All 13 required unit tests were executed on the native host environment using the Unity test framework. All 13 passed successfully in **1.35 seconds**.

```
PS D:\BCA182 Lab\LAB 1 FreeRTOS Multisensor> pio test -e native
Verbosity level can be increased via '-v, -vv, or -vvv' option
Collected 1 tests

Processing test_logic in native environment
-----
Building...
Testing...
test\test_logic\test_logic.c:101: test_temperature_below_lower_limit_is_low [PASSED]
test\test_logic\test_logic.c:102: test_temperature_exactly_lower_limit_is_normal [PASSED]
test\test_logic\test_logic.c:103: test_temperature_normal_value_is_normal [PASSED]
test\test_logic\test_logic.c:104: test_temperature_exactly_upper_limit_is_normal [PASSED]
test\test_logic\test_logic.c:105: test_temperature_above_upper_limit_is_high [PASSED]
test\test_logic\test_logic.c:108: test_next_from_temperature_is_humidity [PASSED]
test\test_logic\test_logic.c:109: test_next_from_motion_wraps_to_temperature [PASSED]
test\test_logic\test_logic.c:110: test_previous_from_humidity_is_temperature [PASSED]
test\test_logic\test_logic.c:111: test_previous_from_temperature_wraps_to_motion [PASSED]
test\test_logic\test_logic.c:114: test_active_without_timeout_stays_active [PASSED]
test\test_logic\test_logic.c:115: test_active_after_timeout_becomes_inactive [PASSED]
test\test_logic\test_logic.c:116: test_inactive_without_motion_stays_inactive [PASSED]
test\test_logic\test_logic.c:117: test_inactive_with_motion_becomes_active [PASSED]
----- native:test_logic [PASSED] Took 1.35 seconds -----

===== SUMMARY =====
Environment  Test      Status  Duration
-----
native      test_logic PASSED   00:00:01.352
```

Figure 2: Execution capture of pio test -e native verifying all 13 unit tests passed under the Unity test runner.

```
Processing test_logic in native environment
-----
test\test_logic\test_logic.c:101: test_temperature_below_lower_limit_is_low [PASSED]
test\test_logic\test_logic.c:102: test_temperature_exactly_lower_limit_is_normal [PASSED]
test\test_logic\test_logic.c:103: test_temperature_normal_value_is_normal [PASSED]
test\test_logic\test_logic.c:104: test_temperature_exactly_upper_limit_is_normal [PASSED]
test\test_logic\test_logic.c:105: test_temperature_above_upper_limit_is_high [PASSED]
test\test_logic\test_logic.c:108: test_next_from_temperature_is_humidity [PASSED]
test\test_logic\test_logic.c:109: test_next_from_motion_wraps_to_temperature [PASSED]
test\test_logic\test_logic.c:110: test_previous_from_humidity_is_temperature [PASSED]
test\test_logic\test_logic.c:111: test_previous_from_temperature_wraps_to_motion [PASSED]
test\test_logic\test_logic.c:114: test_active_without_timeout_stays_active [PASSED]
test\test_logic\test_logic.c:115: test_active_after_timeout_becomes_inactive [PASSED]
test\test_logic\test_logic.c:116: test_inactive_without_motion_stays_inactive [PASSED]
test\test_logic\test_logic.c:117: test_inactive_with_motion_becomes_active [PASSED]
===== 13 test cases: 13 succeeded in 00:00:01.352 =====
```

## 5.2 Functional Verification in Wokwi (FT-01 to FT-10)

| Test ID | Requirement | Stimulus / Input                          | Expected Result                          | Actual Observed Behavior                               | Verdict |
|---------|-------------|-------------------------------------------|------------------------------------------|--------------------------------------------------------|---------|
| FT-01   | FR-01       | Set DHT22 temperature to 28.0 °C in Wokwi | Displayed temperature updates to 28.0 °C | Serial log: [Sensor] Temp: 28.0 °C; OLED shows 28.0 °C | PASS    |

| Test ID | Requirement | Stimulus / Input                             | Expected Result                                       | Actual Observed Behavior                                      | Verdict |
|---------|-------------|----------------------------------------------|-------------------------------------------------------|---------------------------------------------------------------|---------|
| FT-02   | FR-02       | Set DHT22 humidity to 65.0 % in Wokwi        | Displayed humidity updates to 65.0 %                  | Serial log: [Sensor] Hum: 65.0 %; OLED shows 65.0 %           | PASS    |
| FT-03   | FR-03       | Adjust LDR lux slider to 300 lux             | Displayed light percentage changes                    | Serial log: LDR: 17% -> 6% -> 99% as slider moved             | PASS    |
| FT-04   | FR-06       | Rotate rotary encoder clockwise              | View advances: Temp -> Hum -> Light -> Motion -> Temp | Display switches pages in forward sequence; serial confirms   | PASS    |
| FT-05   | FR-06       | Rotate rotary encoder counterclockwise       | View reverses: Temp -> Motion -> Light -> Hum -> Temp | Display switches pages in reverse sequence with full wrap     | PASS    |
| FT-06   | FR-07       | Set temperature to 32.5 °C (> 30.0°C)        | Alarm activates; buzzer turns ON; OLED shows warning  | Buzzer activates (1 kHz tone); OLED displays ! TEMP ALARM !   | PASS    |
| FT-07   | FR-07       | Return temperature to 25.0 °C                | Alarm clears; buzzer turns OFF                        | Buzzer stops immediately; ! TEMP ALARM ! warning disappears   | PASS    |
| FT-08   | FR-04, 08   | Click PIR sensor in Wokwi                    | Motion detected; system remains ACTIVE                | Serial: [PIR] Motion detected; Motion: DETECTED shown         | PASS    |
| FT-09   | FR-09       | Leave PIR untouched for 15 seconds           | System transitions to INACTIVE; OLED powers off       | Serial: [System] State transitioned to: INACTIVE; OLED blanks | PASS    |
| FT-10   | FR-10       | Trigger PIR or rotate encoder while INACTIVE | System wakes to ACTIVE; OLED powers on immediately    | Serial: [System] State transitioned to: ACTIVE; OLED restored | PASS    |

## 5.3 Deliberate FreeRTOS Fault Experiments

### Fault Experiment 1: Remove Blocking Delay from a Task

- **Procedure:** The blocking delay `vTaskDelayUntil(&lastWakeTime;, ...)` was temporarily removed from `SensorTask` (Priority 2), turning it into a continuous busy loop.
- **Observed Result:** `DisplayTask` (Priority 1) experienced complete CPU starvation. The OLED stopped refreshing entirely, and the PC13 heartbeat LED stopped toggling. However, `InputTask` and `MotionTask` (Priority 3) continued to execute whenever an interrupt occurred because their priority is higher than `SensorTask`.
- **Engineering Explanation:** In a priority-based preemptive RTOS, the scheduler always allocates CPU time to the highest-priority Ready task. A Priority 2 task that never blocks monopolizes 100% of available CPU time, starving all lower-priority tasks.
- **Resolution:** `vTaskDelayUntil()` was restored immediately.

## Fault Experiment 2: Inappropriate Task Priority Assignment

- **Procedure:** DisplayTask was artificially promoted to Priority 4 (above InputTask and MotionTask).
- **Observed Result:** Rotary encoder response became noticeably sluggish during OLED framebuffer updates. When rapidly rotating the encoder knob, step pulses were delayed and occasionally dropped because the heavy I2C transmission loop monopolized the processor.
- **Engineering Explanation:** Priority denotes *scheduling urgency*, not importance. UI display operations are computationally heavy and have high latency tolerance, whereas quadrature encoder decoding requires sub-millisecond response. Assigning high priority to UI violates rate-monotonic and urgency-based design.
- **Resolution:** DisplayTask was restored to Priority 1.

## Fault Experiment 3: Remove Serial Mutex (serialMutex)

- **Procedure:** Mutex acquisition (Log\_Begin() / Log\_End()) was temporarily commented out, allowing SensorTask and InputTask to write directly to USART1 concurrently.
- **Observed Result:** When the encoder was turned while SensorTask was printing its telemetry line, the output stream became corrupted:

[Sensor] Te[Encoder] View selected: Humiditmp: 25.0 C | Hum:50.0 %

- **Engineering Explanation:** USART1->DR is a non-reentrant hardware resource. Without mutual exclusion, context switches occurring mid-string interleave byte streams, creating corrupt logs that break automated monitoring tools.
- **Resolution:** serialMutex was restored.

## 6. Static Code Analysis (pio check)

Static code analysis was executed using **PlatformIO Check** with cppcheck.

### 6.1 Findings and Corrective Actions

| Finding ID | File / Line      | Severity    | Root Cause                                                    | Resolution Applied                                                        |
|------------|------------------|-------------|---------------------------------------------------------------|---------------------------------------------------------------------------|
| CHK-01     | dht22.c:31       | Style (Low) | Unsigned expression ticks checked against unsigned difference | Added / cppcheck-suppress unsignedLessThanZero / (wrap-safe elapsed time) |
| CHK-02     | dht22.c:66       | Style (Low) | Unsigned expression limit checked against unsigned difference | Added / cppcheck-suppress unsignedLessThanZero / (wrap-safe elapsed time) |
| CHK-03     | diagnostics.c:36 | Style (Low) | Parameter stacked_regs passed without const qualifier         | Updated function signature to const uint32_t *stacked_regs                |

| Finding ID | File / Line            | Severity    | Root Cause                                         | Resolution Applied                                                         |
|------------|------------------------|-------------|----------------------------------------------------|----------------------------------------------------------------------------|
| CHK-04     | main.c:116             | Style (Low) | Parameter pcTaskName in FreeRTOS hook lacked const | Added / cppcheck-suppress constParameterPointer / to maintain FreeRTOS API |
| CHK-05     | stm32f1xx_hal_msp.c:9  | Style (Low) | Parameter hi2c in HAL callback lacked const        | Added / cppcheck-suppress constParameterPointer / to maintain ST HAL API   |
| CHK-06     | stm32f1xx_hal_msp.c:25 | Style (Low) | Parameter hi2c in HAL callback lacked const        | Added / cppcheck-suppress constParameterPointer / to maintain ST HAL API   |
| CHK-07     | stm32f1xx_hal_msp.c:32 | Style (Low) | Parameter huart in HAL callback lacked const       | Added / cppcheck-suppress constParameterPointer / to maintain ST HAL API   |
| CHK-08     | stm32f1xx_hal_msp.c:52 | Style (Low) | Parameter huart in HAL callback lacked const       | Added / cppcheck-suppress constParameterPointer / to maintain ST HAL API   |

## 6.2 Final Analysis Result

```

Checking bluepill_f103c8 > cppcheck (platform: ststm32; board: bluepill_f103c8; framework: stm32cube)
-----
No defects found
===== [PASSED] Took 1.84 seconds =====

```

**0 high, 0 medium, 0 low defects.**

## 7. Engineering Discussion

### 7.1 Wokwi Simulation vs. Physical Hardware Deep-Dive

During development, a critical architectural challenge was diagnosed: Wokwi's virtual Cortex-M3 emulator fails during standard FreeRTOS task startup when returning from SVC 0 using EXC\_RETURN (0xFFFFFDD), halting in prvTaskExitError().

To ensure rock-solid simulation reliability:

- Thread-Mode Bootstrap:** prvPortStartFirstTask() was adapted to configure PSP directly, switch CONTROL to Thread mode on PSP, and branch directly into prvTaskBootstrap, bypassing the emulator unstacking fault.
- Context Switching:** portYIELD() was mapped to vPortYieldDirect(), performing a naked Thread-mode context switch between task stacks.
- Tick Timer:** TIM3 was configured at 20 Hz (50 ms tick) to advance the RTOS scheduler and uwTick smoothly without burdening the host browser's JavaScript engine.

*Technical Attribution:* The Thread-mode direct context-switching and TIM3 tick compatibility patch was adapted from the open-source implementation by Djaver Hassan (credit: Ni-ear).

### Differences on Physical Hardware:

- On physical STM32F103 silicon, the standard FreeRTOS port works natively: SVC #0 unrolls the stack into Thread mode via hardware exception return, and PendSV handles all preemptive context switches at priority 15.
- The physical hardware uses the internal SysTick timer at 1000 Hz (1 ms tick resolution) rather than TIM3.
- Physical I2C requires external 4.7 k $\Omega$  pull-up resistors on PB6/PB7 to 3.3V, and ADC1 requires executing HAL\_ADCEX\_Calibration\_Start() upon boot.

## 7.2 Memory Allocation & Sizing

- **Total SRAM Available:** 20 KB (20,480 bytes).
- **FreeRTOS Heap (heap\_4):** Allocated 10 KB (10,240 bytes).
- **Task Stacks:** Each application task was allocated 256 words (1,024 bytes), with the Idle task allocated 128 words (512 bytes).
- **RAM Breakdown:**
  - Global Static Variables (.data + .bss): 1,916 bytes (including 1024-byte OLED framebuffer).
  - FreeRTOS Heap: 10,240 bytes.
  - Main Stack (MSP): 8,324 bytes.
- Total RAM Utilization: **59.4%** (12,156 bytes). Zero heap exhaustion or stack overflows occurred.

## 8. Conclusion

---

This laboratory activity successfully designed, implemented, and verified a concurrent real-time room monitoring node on the STM32 Blue Pill using PlatformIO, the STM32Cube framework, and FreeRTOS.

### Key Accomplishments:

1. Reconstructed a complete concurrent embedded system adhering strictly to the **STM32Cube framework** (zero Arduino abstractions).
2. Implemented all **6 required FreeRTOS tasks** with justified priorities and verified inter-task communication via queues, a recursive mutex, and an event group.
3. Implemented deterministic, drift-free sensor sampling via vTaskDelayUntil().
4. Achieved **100% pass rate across 13 automated unit tests** on the Unity native framework.
5. Achieved **0 defects in static code analysis** with PlatformIO Check (cppcheck).
6. Successfully adapted and documented the firmware for Wokwi virtual simulation constraints while maintaining full architectural fidelity for physical deployment.